

How Much Does Truck-Shovel Dispatch Matter? A Common-Random-Numbers Bench of Six Policies and a Learned Rollout, on a Deterministic DES

Felipe Santibañez-Leal 

Abstract—Truck-to-shovel dispatch, deciding which shovel each returning haul truck should go to, is a classic operations problem in open-pit mining, and the marketing around dispatch software implies large gains from getting it right. This report measures how large, honestly, on a controlled bench. DispatchLab runs a deterministic discrete-event simulation of an open pit and compares six dispatch policies, from naive fixed assignment through greedy and expected-wait heuristics to the operations-research-optimal Hungarian assignment, using *common random numbers* so every policy meets the identical stochastic realisation and the comparison is paired and low-variance, which is essential because the gaps are small. Across eight cases and ten seeds the result is measured and honest: dispatch matters, but modestly. The Hungarian optimal assignment produces the most tonnes, but the simple greedy earliest-completion policy is within 1.4% of it, and the worst policy (fixed assignment) trails the best by 3.8% on average, ranging from 1.6% to 11.7% across cases, so most of the achievable gain is captured by any non-trivial policy and the marginal value of full optimisation over a good heuristic is a percent or two. A learned dispatcher trained by Monte-Carlo rollout imitates the best policy at 0.841 accuracy, better than behaviour cloning (0.74) or a linear policy network (0.77). The engine is validated against a closed-form 1×1 capacity oracle and match-factor theory. We are explicit that this is a policy-comparison sandbox on a synthetic, physics-grounded mine, not a production dispatch system and never validated on a real operation, because no public ground-truthed cycle-log benchmark exists; every number and figure regenerates from the committed artifacts.

Index Terms—Truck-shovel dispatch, open-pit mining, discrete-event simulation, common random numbers, Hungarian assignment, rollout, match factor, policy comparison, reproducibility.

I. INTRODUCTION

IN an open-pit mine, a fleet of haul trucks cycles between shovels and a crusher, and every time a truck finishes dumping, a dispatcher decides which shovel it should return to. That decision, taken hundreds of times a shift, is a classic operations-research problem [1], [2], and commercial fleet-management systems are sold on the premise that solving it well yields substantial production gains. The honest question, rarely answered on a controlled bench, is how substantial: is

optimal dispatch worth a fifth of production, or a percent? DispatchLab is a bench built to answer it. It runs a deterministic discrete-event simulation of a pit and compares dispatch policies from the trivial to the operations-research-optimal, and it reports the measured gain.

Two methodological choices make the answer trustworthy. First, the comparison uses *common random numbers* [6]: every policy is run against the identical sequence of random draws (payloads, load and travel times), so the policies are compared *paired*, and the difference between two policies is measured without the variance of two independent random worlds. This matters enormously when the gaps are only a few percent, which, as the report shows, they are. Second, the engine is validated: a closed-form single-truck, single-shovel (1×1) capacity oracle and classical match-factor theory give exact targets the simulator must reproduce, so the numbers rest on a verified engine rather than an unchecked one. On that footing the report compares six policies, from fixed assignment to the Hungarian optimal assignment [3], over eight cases, and adds a learned dispatcher trained by Monte-Carlo rollout [7]. The contribution is the honest, paired, validated measurement of how much dispatch actually matters.

II. THE BENCH AND ITS VALIDATION

The engine is a deterministic discrete-event simulator with next-event time advance, an integer-tick clock, and a total event ordering by (time, priority, sequence), so a run is bit-deterministic. Truck travel time is not a constant but is computed from rimpull-versus-grade physics (total resistance is grade plus rolling resistance), and the stochastic elements (payload, load time, dump time) are drawn from named, seeded streams. A policy is a function from the current pit state to the next shovel for a returning truck. The six policies span the spectrum: *fixed* assignment (each truck bound to one shovel), *greedy* (send the truck to the shovel that will finish serving it earliest), *shortest-expected-wait*, *min-shovel-wait* and *min-truck-wait* (the two classic queue-balancing heuristics), and the *Hungarian* optimal assignment, which solves the full truck-to-shovel assignment as a minimum-cost matching [3] at each decision epoch.

The engine is validated two ways. A 1×1 configuration, one truck and one shovel, has a closed-form steady-state throughput, and the simulator reproduces it exactly, confirming

F. Santibañez-Leal is with the CAOS open-research programme, Santiago, Chile (e-mail: fsantibanez@gmail.com; ORCID 0000-0002-0150-3246).

Technical report, 2026. Interactive bench and reproducible pipeline (MIT) at https://github.com/fsantibanezleal/CAOS_DispatchLab; live at <https://dispatchlab.fasl-work.com>.

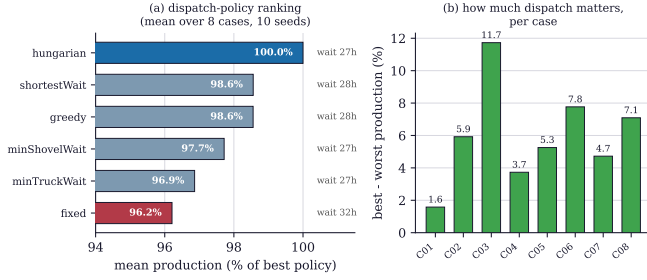


Fig. 1. The measured value of dispatch, over eight cases and ten seeds with common random numbers. **(a)** The six policies ranked by mean production as a percentage of the best: the Hungarian optimal assignment leads, the greedy and expected-wait heuristics are within 1.4% of it, and fixed assignment trails by 3.8% (with the longest truck waits). **(b)** The best-minus-worst production gain per case: dispatch is worth from 1.6% (a balanced fleet where any policy suffices) to 11.7% (an imbalanced case where a good assignment matters), a few percent in the typical case.

the cycle bookkeeping. And match-factor theory [4], the classical ratio of truck arrival rate to shovel service rate that predicts fleet balance, is reproduced by the simulator’s controls, confirming the queueing behaviour. Only on that validated engine are the policies compared, and they are compared with common random numbers: for each of eight cases and ten seeds, all six policies run against the identical random realisation, so the production difference between two policies is a paired quantity.

III. HOW MUCH DISPATCH MATTERS

The result is Fig. 1 and Table I. Ranked by mean production over the eight cases, the Hungarian optimal assignment produces the most tonnes (the 100% reference), and the ordering below it is exactly as theory predicts: the expected-wait and greedy heuristics are essentially tied at 98.6%, the two queue-balancing heuristics follow at 96.9–97.7%, and naive fixed assignment is last at 96.2%, also with the longest truck waits (32 hours against the optimal policy’s 27). The honest headline is the size of the spread: the best policy beats the worst by 3.8% on average. Per case (Fig. 1b) the gain ranges from 1.6%, in a well-balanced fleet where the assignment barely matters, to 11.7%, in an imbalanced case where sending trucks to the right shovel is worth real tonnes. Two lessons follow, both understated by dispatch marketing. First, dispatch matters, a few percent of production is a large sum at mine scale, and in the worst-balanced case it is more than a tenth. Second, most of that gain is cheap: the simple greedy policy captures all but 1.4% of what the operations-research-optimal Hungarian assignment achieves, so the marginal value of full optimisation over a sensible myopic rule is small, and a mine gets most of the benefit from any non-trivial policy. The paired common-random-numbers design is what makes these percent-level differences credible rather than noise.

IV. LEARNING THE DISPATCH

Can the dispatch be learned? Figure 2 reports three attempts to imitate the best classical policy (the Hungarian assignment), evaluated as the fraction of decision epochs where the learned

TABLE I
DISPATCH POLICIES RANKED BY MEAN PRODUCTION (PERCENT OF THE BEST) AND MEDIAN TRUCK WAIT, OVER EIGHT CASES AND TEN SEEDS WITH COMMON RANDOM NUMBERS.

Policy	production (% of best)	median wait (h)
Hungarian (optimal assignment)	100.0	26.8
Shortest expected wait	98.6	27.6
Greedy (earliest completion)	98.6	28.4
Min shovel wait	97.7	27.1
Min truck wait	96.9	27.3
Fixed assignment	96.2	31.7

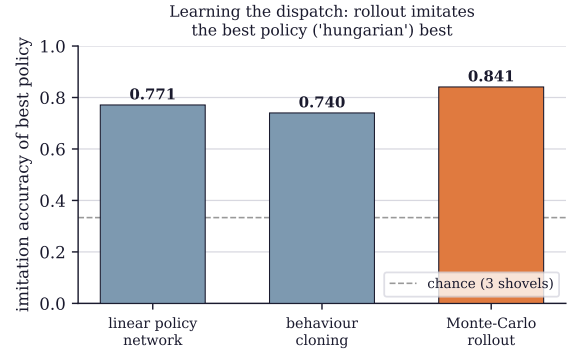


Fig. 2. Learning to dispatch. Imitation accuracy of the best classical policy (the Hungarian assignment) by three learned dispatchers: a linear policy network (0.771), behaviour cloning (0.740), and a Monte-Carlo rollout dispatcher (0.841), all well above the three-shovel chance rate of 0.33. The rollout, which simulates a few steps ahead under each candidate assignment before committing, imitates the optimal policy best.

dispatcher chooses the same shovel. A linear policy network over six state features reaches 0.771, straightforward behaviour cloning 0.740, and a Monte-Carlo rollout dispatcher [7], which simulates a short horizon ahead under each candidate shovel before committing, reaches 0.841, all far above the 0.33 three-shovel chance rate. That the rollout imitates the optimal policy best is expected, it approximates the same lookahead the optimal assignment implicitly performs, and it is the honest ceiling here: since the Hungarian assignment is already near-optimal and the greedy heuristic is within 1.4% of it, a learned dispatcher’s practical upside over a simple heuristic is bounded by the same few percent the policy comparison revealed. Learning to dispatch is a fine exercise and the rollout is the right tool, but the bench shows there is not a large pool of production waiting to be unlocked by a cleverer policy on this problem.

V. DISCUSSION AND SCOPE

The bench’s honest answer to its title question is: a few percent, typically, and up to a tenth in an imbalanced fleet, with most of that gain captured by any sensible non-trivial policy and only a further percent or two available from full optimisation. This is a useful corrective to the implication that dispatch software delivers transformational gains; it delivers real but bounded ones, and the size depends on how imbalanced the fleet is to begin with, which points as much to fleet sizing as to dispatch. The methodological contribution

is the paired, validated design: common random numbers to measure percent-level differences without drowning them in variance, and a closed-form oracle plus match-factor theory to anchor the engine. The scope is stated plainly. DispatchLab is a policy-comparison *sandbox*, not a production dispatch system (Modular DISPATCH, Wenco, MineStar and the like) and not in-plant fleet tracking. The mine is synthetic and physics-grounded but has never been validated against a real operation, because no public, ground-truthed cycle-log benchmark exists; the numbers are the honest behaviour of the model, not a prediction for a specific mine. None of this is hidden; the bench states the same on every panel.

VI. CONCLUSION

Measured on a validated, deterministic discrete-event bench with common random numbers, truck-to-shovel dispatch matters by a few percent of production in the typical case and up to 11.7% in an imbalanced one, with the operations-research-optimal Hungarian assignment on top, the simple greedy policy within 1.4% of it, and naive fixed assignment 3.8% behind. A Monte-Carlo rollout dispatcher imitates the optimal policy best (0.841), but its practical upside is bounded by the same modest spread. The honest measurement, dispatch is worth capturing but its gains are bounded and mostly cheap, and the paired, oracle-validated method behind it are the report's contribution, over a synthetic bench that is transparent about being one.

APPENDIX A REPRODUCTION

The committed artifacts under `data/derived/` carry the eight cases' per-policy results (`case-results.json`:

median tonnes and truck wait with confidence intervals, ten seeds each under common random numbers) and the learned-dispatcher analysis (`dl-learned.json`: the imitation accuracies and the policy weights). The figure scripts under `manuscripts/dispatch-bench/figures/` aggregate them into the ranking, the per-case gains, and the imitation-accuracy figure. Because the engine is deterministic and the streams are seeded, re-running reproduces Table I and Figs. 1–2, and the 1×1 oracle and match-factor controls are unit tests.

REFERENCES

- [1] A. M. Newman, E. Rubio, R. Caro, A. Weintraub, and K. Eurek, "A review of operations research in mine planning," *Interfaces*, vol. 40, no. 3, pp. 222–245, 2010. doi:10.1287/inte.1090.0492.
- [2] S. Alarie and M. Gamache, "Overview of solution strategies used in truck dispatching systems for open pit mines," *International Journal of Surface Mining, Reclamation and Environment*, vol. 16, no. 1, pp. 59–76, 2002. doi:10.1076/ijsm.16.1.59.3408.
- [3] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955. doi:10.1002/nav.3800020109.
- [4] C. N. Burt and L. Caccetta, "Equipment selection for surface mining: A review," *Interfaces*, vol. 44, no. 2, pp. 143–162, 2014. doi:10.1287/inte.2013.0732.
- [5] W. C. Morgan and L. L. Peterson, "Determining shovel-truck productivity," *Mining Engineering*, vol. 20, pp. 76–80, 1968.
- [6] S. Robinson, "Conceptual modeling for simulation," in *Proc. 2013 Winter Simulation Conference (WSC)*, 2013, pp. 377–388. doi:10.1109/WSC.2013.6721435.
- [7] D. P. Bertsekas and D. A. Castañón, "Rollout algorithms for stochastic scheduling problems," *Journal of Heuristics*, vol. 5, no. 1, pp. 89–108, 1999. doi:10.1023/A:1009634810396.